Descripción Ver entrega

**Límite de entrega**: viernes, 25 de abril de 2025, 10:30
**Ficheros requeridos**: prueba.py, networking.py (Descargar)
**Tipo de trabajo**: Individual

## Gestión de Grupos de Interés

Se pide completar el desarrollo de la clase `InterestGroup` en el módulo `networking`. Cada objeto de la clase `InterestGroup` representa un grupo de personas con intereses comunes. Para almacenar los miembros del grupo se usará una **lista encadenada**. Se proporciona la clase interna `Node` que define las características de los nodos de la lista. Los miembros del grupo están representados por sus nombres (tipo `str`).

La clase `InterestGroup` hereda de la clase `ContainerBase`, que define el método `__str__`, permitiendo mostrar la lista de miembros de forma sencilla.

Tu tarea es completar el funcionamiento de la clase `InterestGroup` añadiendo atributos y modificando métodos que crea necesarios con el fin de desarrollar los siguientes métodos y propiedades dentro de la clase `InterestGroup`:

### Métodos y propiedades a desarrollar:

1. **Método** `is_member`
   Devuelve `True` si el nombre pasado por parámetro está en el grupo, `False` en caso contrario.

2. **Método** `add_member`
   Añade un nuevo miembro (nombre pasado por parámetro) al grupo si no estaba previamente. Devuelve `True` si lo añade y `False` si no.

3. **Propiedad** `size`
   Devuelve el número total de miembros del grupo.

4. **Método** `union`
   Devuelve un nuevo grupo con la **unión** de miembros del grupo actual y el grupo recibido por parámetro (sin duplicados).

5. **Método** `remove_member`
   Se le pasa un nombre por parámetro y lo elimina si está en el grupo. Devuelve `True` si lo elimina y `False` si no.

---

### Recomendaciones:

- **Localice los errores** en su código modificando y ejecutando el fichero `prueba.py`. El contenido de este fichero no se evalúa.
- Recuerda que la lista encadenada debe construirse sin usar listas nativas de Python.
- El orden de los miembros en el grupo no es relevante.

---

### Ejemplo de uso:

?

```
group1 = InterestGroup()
group2 = InterestGroup()

group1.add_member("Ana")
group1.add_member("Luis")
group1.add_member("Marta")

group2.add_member("Luis")
group2.add_member("Carlos")
group2.add_member("Elena")

print(group1)
# ➡️[Ana]->[Luis]->[Marta]->None

print(group1.is_member("Luis")) # True
print(group1.size) # 3

union_group = group1.union(group2)
print(union_group)
# ➡️[Ana]->[Luis]->[Marta]->[Carlos]->[Elena]->None

union_group.remove_member("Elena")
print(union_group)
# ➡️[Ana]->[Luis]->[Marta]->[Carlos]->None
```

## English version

## 👫 Stakeholder Group Management

You are asked to complete the development of the `InterestGroup` class in the `networking` module. Each object of the `InterestGroup` class represents a group of people with common interests. To store the group members, a linked list will be used. An inner class `Node` is provided, which defines the characteristics of the list nodes. Group members are represented by their names (type `str`).

The `InterestGroup` class inherits from the `ContainerBase` class, which defines the `__str__` method, allowing for easy display of the list of members.

Your task is to complete the functionality of the `InterestGroup` class by adding attributes and modifying methods as needed in order to implement the following methods and properties within the class:

### ✅ Methods and properties to implement:

- **Method** `is_member`
  Returns `True` if the name passed as a parameter is in the group, `False` otherwise.
- **Method** `add_member`
  Adds a new member (name passed as a parameter) to the group if it was not already present. Returns `True` if it adds the member, and `False` if it doesn't.
- **Property** `size`
  Returns the total number of members in the group.
- **Method** `union`
  Returns a new group containing the union of members from the current group and the group passed as a parameter (without duplicates).
- **Method** `remove_member`
  Takes a name as a parameter and removes it from the group if it is present. Returns `True` if it is removed, and `False` if not.

### 🧠 Recommendations:

- Identify errors in your code by modifying and executing the `prueba.py` file. The content of this file will not be evaluated.
- Remember that the linked list must be implemented without using Python's built-in list structures.
- The order of members in the group is not relevant.

### 🔄 Usage example:

```
group1 = InterestGroup()
group2 = InterestGroup()

group1.add_member("Ana")
group1.add_member("Luis")
group1.add_member("Marta")

group2.add_member("Luis")
group2.add_member("Carlos")
group2.add_member("Elena")

print(group1)
# ➡️[Ana]->[Luis]->[Marta]->None

print(group1.is_member("Luis")) # True
print(group1.size) # 3

union_group = group1.union(group2)
print(union_group)
# ➡️[Ana]->[Luis]->[Marta]->[Carlos]->[Elena]->None

union_group.remove_member("Elena")
print(union_group)
# ➡️[Ana]->[Luis]->[Marta]->[Carlos]->None
```

©2012-2015 - Laurent Pointal Mémento v2.0.6
License Creative Commons Attribution 4

# Python 3 Cheat Sheet

*Latest version on :*
*https://perso.limsi.fr/pointal/python:memento*

## Base Types

*integer, float, boolean, string, bytes*

```
int 783  0 -192  0b010 0o642 0xF3
float 9.23 0.0  -1.7e-6
bool True False
str "One\nTwo"
    'I\'m'
bytes b"toto\xfe\775"
```

*zero* *binary* *octal* *hexa*

*escaped new line* *escaped '* *escaped tab* *hexadecimal* *octal* *$\times 10^{-6}$*

*Multiline string:* `"""X\tY\tZ 1\t2\t3"""`

*immutables*

## Container Types

- **ordered sequences**, fast index access, repeatable values

```
list [1,5,9]   ["x",11,8.9]   ["mot"]   []
tuple (1,5,9)  11,"y",7.4     ("mot",)  ()
str bytes
```

*Non modifiable values (immutables)* *expression with only comas → `tuple`* *(ordered sequences of chars / bytes)*

- **key containers**, no *a priori* order, fast key access, each key is unique

**dictionary** `dict {"key":"value"}` `dict(a=3,b=4,k="v")` `{}`
*(key/value associations)* `{1:"one",3:"three",2:"two",3.14:"π"}`

**collection** `set {"key1","key2"}` `{1,9,3,0}` `set()`

*keys=hashable values (base types, immutables…)* `frozenset` *immutable set* *empty*

## Identifiers

*for variables, functions, modules, classes… names*

`a…zA…Z_` followed by `a…zA…Z_0…9`
- diacritics allowed but should be avoided
- language keywords forbidden
- lower/UPPER case discrimination

☺ `a toto x7 y_max BigOne`
☹ `~~By and for~~`

## Variables assignment

`=`

assignment ⇔ **binding** of a *name* with a *value*
1) *evaluation of right side expression value*
2) *assignment in order with left side names*

```
x=1.2+8+sin(y)
a=b=c=0      assignment to same value
y,z,r=9.2,-7.6,0   multiple assignments
a,b=b,a      values swap
a,*b=seq     unpacking of sequence in
*a,b=seq     item and list
x+=3         increment ⇔ x=x+3
x-=2         decrement ⇔ x=x-2
x=None       « undefined » constant value
del x        remove name x
```

and `*=` `/=` `%=` …

## Conversions

`type(expression)`

```
int("15") → 15
int("3f",16) → 63          can specify integer number base in 2nd parameter
int(15.56) → 15            truncate decimal part
float("-11.24e8") → -1124000000.0
round(15.56,1)→15.6        rounding to 1 decimal (0 decimal → integer number)
bool(x)  False for null x, empty container x , None or False x ; True for other x
str(x)→ "…"   representation string of x for display (cf. formatting on the back)
chr(64)→'@'  ord('@')→64   code ↔ char
repr(x)→ "…"  literal representation string of x
bytes([72,9,64]) → b'H\t@'
list("abc") → ['a','b','c']
dict([(3,"three"),(1,"one")]) → {1:'one',3:'three'}
set(["one","two"]) → {'one','two'}
separator str and sequence of str → assembled str
  ':'.join(['toto','12','pswd']) → 'toto:12:pswd'
str splitted on whitespaces → list of str
  "words with  spaces".split() → ['words','with','spaces']
str splitted on separator str → list of str
  "1,4,8,2".split(",") → ['1','4','8','2']
sequence of one type → list of another type (via list comprehension)
  [int(x) for x in ('1','29','-3')] → [1,29,-3]
```

## Sequence Containers Indexing

*for lists, tuples, strings, bytes…*

| | | | | | |
|---|---|---|---|---|---|
| *negative index* | −5 | −4 | −3 | −2 | −1 |
| *positive index* | 0 | 1 | 2 | 3 | 4 |
| `lst=[10,` | `20,` | `30,` | `40,` | `50]` | |
| *positive slice* | 0 | 1 | 2 | 3 | 4 | 5 |
| *negative slice* | −5 | −4 | −3 | −2 | −1 | |

**Items count**
`len(lst)→5`

**index from 0**
(here from 0 to 4)

Individual access to **items** via `lst[index]`
`lst[0]→10` ⇒ *first one* `lst[1]→20`
`lst[-1]→50` ⇒ *last one* `lst[-2]→40`

*On mutable sequences (`list`), remove with `del lst[3]` and modify with assignment `lst[4]=25`*

Access to **sub-sequences** via `lst[start slice:end slice:step]`

```
lst[:-1]→[10,20,30,40]   lst[::-1]→[50,40,30,20,10]   lst[1:3]→[20,30]   lst[:3]→[10,20,30]
lst[1:-1]→[20,30,40]     lst[::-2]→[50,30,10]         lst[-3:-1]→[30,40] lst[3:]→[40,50]
lst[::2]→[10,30,50]      lst[:]→[10,20,30,40,50] shallow copy of sequence
```

*Missing slice indication → from start / up to end.*
*On mutable sequences (`list`), remove with `del lst[3:5]` and modify with assignment `lst[1:4]=[15,25]`*

## Boolean Logic

Comparisons : `< > <= >= == !=` (*boolean results*) ≤ ≥ = ≠

`a and b` logical and *both simultaneously*

`a or b` logical or *one or other or both*

pitfall : `and` *and* `or` *return value of* `a` *or of* `b` *(under shortcut evaluation).*
⇒ *ensure that* `a` *and* `b` *are booleans.*

`not a` logical not

`True` `False` True and False constants

## Statements Blocks

*parent statement :*
*statement block 1…*
⋮
*parent statement :*
*statement block2…*
⋮
*next statement after block 1*

indentation

*configure editor to insert 4 spaces in place of an indentation tab.*

## Modules/Names Imports

*module `truc`⇔file `truc.py`*

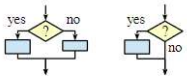
```
from monmod import nom1,nom2 as fct
  →direct access to names, renaming with as
import monmod →access via monmod.nom1 …
```

modules and packages searched in *python path* (cf `sys.path`)

## Conditional Statement

*statement block executed only if a condition is true*

```
if logical condition :
    statements block
```




Can go with several *elif*, *elif*… and only one final *else*. Only the block of first true condition is executed.

```
if age<=18:
  state="Kid"
elif age>65:
  state="Retired"
else:
  state="Active"
```

*with a var `x`:*
`if bool(x)==True:` ⇔ `if x:`
`if bool(x)==False:` ⇔ `if not x:`

## Maths

*floating numbers… approximated values* *angles in radians*

Operators: `+ - * / // % **`
Priority `(…)` × ÷ ↑ ↑ $a^b$ integer ÷ ÷ remainder

`@` → matrix × *python3.5+`numpy`*

```
(1+5.3)*2→12.6
abs(-3.2)→3.2
round(3.57,1)→3.6
pow(4,3)→64.0
```

*usual order of operations*

```
from math import sin,pi…
sin(pi/4)→0.707…
cos(2*pi/3)→-0.4999…
sqrt(81)→9.0  √
log(e**2)→2.0
ceil(12.5)→13
floor(12.5)→12
```

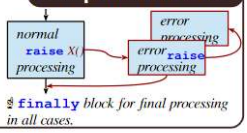
modules `math`, `statistics`, `random`, `decimal`, `fractions`, `numpy`, *etc. (cf. doc)*

## Exceptions on Errors

Signaling an error:
`raise` *ExcClass(…)*

Errors processing:

```
try:
    normal procesising block
except Exception as e:
    error processing block
```




`finally` *block for final processing in all cases.*

## Conditional Loop Statement

*statements block executed* ***as long as*** *condition is true*

```
while logical condition:
    statements block
```

```
s = 0
i = 1
while i <= 100:
    s = s + i**2
    i = i + 1
print("sum:",s)
```

*initializations* ***before*** *the loop*

*condition with a least one variable value (here* `i`*)*

*make condition variable change !*

*Algo:* $s = \sum_{i=1}^{i=100} i^2$

*beware of infinite loops!*

## Loop Control

| | |
|---|---|
| `break` | *immediate exit* |
| `continue` | *next iteration* |

`else` *block for* ***normal*** *loop exit.*

## Iterative Loop Statement

*statements block executed* ***for each*** *item of a container or iterator*

```
for var in sequence:
    statements block
```

Go over sequence's **values**

```
s = "Some text"
cnt = 0
for c in s:
    if c == "e":
        cnt = cnt + 1
print("found",cnt,"'e'")
```

*initializations* ***before*** *the loop*

*loop variable,* ***assignment managed by*** `for` *statement*

*Algo: count number of* `e` *in the string.*

loop on dict/set ⇔ loop on keys sequences

use *slices* to loop on a subset of a sequence

Go over sequence's **index**

- modify item at index
- access items around index (before / after)

```
lst = [11,18,9,12,23,4,17]
lost = []
for idx in range(len(lst)):
    val = lst[idx]
    if val > 15:
        lost.append(val)
        lst[idx] = 15
print("modif:",lst,"-lost:",lost)
```

*Algo: limit values greater than 15, memorizing of lost values.*

*good habit : don't modify loop variable*

Go simultaneously over sequence's **index** and **values**:

```
for idx,val in enumerate(lst):
```

## Display

```
print("v=",3,"cm :",x,",",y+4)
```

items to display : literal values, variables, expressions

`print` options:

- `sep=" "` — items separator, default space
- `end="\n"` — end of print, default new line
- `file=sys.stdout` — print to file, default standard output

## Input

```
s = input("Instructions:")
```

`input` always returns a **string**, convert it to required type (cf. boxed *Conversions* on the other side).

## Generic Operations on Containers

`len(c)` → items count

`min(c)` `max(c)` `sum(c)`

`sorted(c)` → `list` sorted *copy*

`val in c` → boolean, membership operator `in` (absence `not in`)

`enumerate(c)` → *iterator* on (index, value)

`zip(c1,c2...)` → *iterator* on tuples containing $c_i$ items at same index

`all(c)` → `True` if **all** `c` items evaluated to true, else `False`

`any(c)` → `True` if **at least one** item of `c` evaluated true, else `False`

*Note: For dictionaries and sets, these operations use* ***keys****.*

*Specific to* ***ordered sequences containers*** *(lists, tuples, strings, bytes...)*

`reversed(c)` → *inversed iterator* `c*5`→ duplicate `c+c2`→ concatenate

`c.index(val)` → *position* `c.count(val)` → *events count*

```
import copy
copy.copy(c) → shallow copy of container
copy.deepcopy(c) → deep copy of container
```

## Integer Sequences

`range([start,] end [,step])`

`start` default 0, `end` not included in sequence, `step` signed, default 1

`range(5)` → 0 1 2 3 4 `range(2,12,3)` → 2 5 8 11

`range(3,8)` → 3 4 5 6 7 `range(20,5,-5)` → 20 15 10

`range(len(seq))` → *sequence of index of values in seq*

*range provides an immutable sequence of int constructed as needed*

## Operations on Lists

modify original list

| | |
|---|---|
| `lst.append(val)` | add item at end |
| `lst.extend(seq)` | add sequence of items at end |
| `lst.insert(idx, val)` | insert item at index |
| `lst.remove(val)` | remove first item with value *val* |
| `lst.pop([idx])` → *value* | remove & return item at index *idx* (default last) |
| `lst.sort()` `lst.reverse()` | sort / reverse liste *in place* |

## Function Definition

```
def fct(x,y,z):
    """documentation"""
    # statements block, res computation, etc.
    return res
```

function name (identifier)

named parameters

result value of the call, if no computed result to return: `return None`

parameters and all variables of this block exist only *in* the block and *during* the function call (think of a "black box")

Advanced: `def fct(x,y,z,*args,a=3,b=5,**kwargs):`

**args variable positional arguments (→`tuple`), default values,*

***kwargs variable named arguments (→`dict`)*

## Operations on Dictionaries

| | |
|---|---|
| `d[key]=value` | `d.clear()` |
| `d[key]` → *value* | `del d[key]` |

`d.update(d2)` — update/add associations

`d.keys()` `d.values()` `d.items()` → *iterable views on keys/values/associations*

`d.pop(key[,default])` → *value*

`d.popitem()` → *(key,value)*

`d.get(key[,default])` → *value*

`d.setdefault(key[,default])` → *value*

## Operations on Sets

Operators:

- `|` → union (vertical bar char)
- `&` → intersection
- `-` `^` → difference/symmetric diff.
- `<` `<=` `>` `>=` → inclusion relations

*Operators also exist as methods.*

`s.update(s2)` `s.copy()`

`s.add(key)` `s.remove(key)`

`s.discard(key)` `s.clear()`

`s.pop()`

## Function Call

```
r = fct(3,i+2,2*i)
```

*storage/use of returned value*

*one argument per parameter*

this is the use of function name *with parentheses* which does the call

*Advanced:*
**sequence*
***dict*

## Operations on Strings

`s.startswith(prefix[,start[,end]])`

`s.endswith(suffix[,start[,end]])` `s.strip([chars])`

`s.count(sub[,start[,end]])` `s.partition(sep)` → *(before,sep,after)*

`s.index(sub[,start[,end]])` `s.find(sub[,start[,end]])`

`s.is...()` *tests on chars categories (ex.* `s.isalpha()`*)*

`s.upper()` `s.lower()` `s.title()` `s.swapcase()`

`s.casefold()` `s.capitalize()` `s.center([width,fill])`

`s.ljust([width,fill])` `s.rjust([width,fill])` `s.zfill([width])`

`s.encode(encoding)` `s.split([sep])` `s.join(seq)`

## Files

*storing data on disk, and reading it back*

```
f = open("file.txt","w",encoding="utf8")
```

**variable** for operations

**name** of file on disk (+path...)

opening **mode**
- `'r'` read
- `'w'` write
- `'a'` append
- ...`'+'` `'x'` `'b'` `'t'`

**encoding** of chars for *text files*: `utf8` `ascii` `latin1` ...

cf. modules `os`, `os.path` and `pathlib`

**writing**

`f.write("coucou")`

`f.writelines(list of lines)`

**reading**

*read empty string if end of file*

`f.read([n])` → *next chars* — *if n not specified, read up to end !*

`f.readline()` → *next line*

`f.readline()` → `list` *of next lines*

*text mode* `t` *by default (read/write* `str`*), possible binary mode* `b` *(read/write* `bytes`*). Convert from/to required type !*

`f.close()` — dont forget to **close the file** after use !

`f.flush()` write cache

`f.truncate([size])` resize

*reading/writing progress sequentially in the file, modifiable with:*

`f.tell()`→*position*

`f.seek(position[,origin])`

Very common: opening with a guarded block (automatic closing) and reading loop on lines of a text file:

```
with open(...) as f:
    for line in f :
        # processing of line
```

## Formatting

```
"modele{} {} {}".format(x,y,r) → str
```

formating directives

values to format

"{*selection* : *formatting* ! *conversion*}"

- **Selection** :
  - `2`
  - `nom`
  - `0.nom`
  - `4[key]`
  - `0[2]`

Examples:

```
"{:+2.3f}".format(45.72793)
→'+45.728'
"{1:>10s}".format(8,"toto")
→'      toto'
"{x!r}".format(x="I'm")
→'"I\'m"'
```

- **Formatting** :

*fill char* *alignment* *sign* *mini width* . *precision~maxwidth* *type*

`< > ^ =` `+ - space` `0` at start for filling with 0

integer: `b` binary, `c` char, `d` decimal (default), `o` octal, `x` or `X` hexa...

float: `e` or `E` exponential, `f` or `F` fixed point, `g` or `G` appropriate (default),

string: `s` ... `%` percent

- **Conversion** : `s` (readable text) or `r` (literal representation)

## Ficheros requeridos

**prueba.py**

```python
from networking import InterestGroup

# Creamos dos grupos
group1 = InterestGroup()
group2 = InterestGroup()

print("Inicialmente:")
print(group1)  # ➡️ None
print(group2)  # ➡️ None

# Añadimos miembros al primer grupo
group1.add_member("Ana")
group1.add_member("Luis")
group1.add_member("Marta")

print("\nGrupo 1 tras añadir miembros:")
print(group1)  # ➡️ [Marta]->[Luis]->[Ana]->None

# Añadimos miembros al segundo grupo
group2.add_member("Luis")
group2.add_member("Carlos")
group2.add_member("Elena")

print("\nGrupo 2 tras añadir miembros:")
print(group2)  # ➡️ [Elena]->[Carlos]->[Luis]->None

# Comprobamos si ciertos miembros están en los grupos
print("\nComprobación de miembros:")
print("Luis en grupo1:", group1.is_member("Luis"))  # True
print("Carlos en grupo1:", group1.is_member("Carlos"))  # False
print("Elena en grupo2:", group2.is_member("Elena"))  # True

# Tamaño de los grupos
print("\nTamaño de los grupos:")
print("Grupo1:", group1.size)  # 3
print("Grupo2:", group2.size)  # 3

# Unión de grupos
union_group = group1.union(group2)
print("\nUnión de grupo1 y grupo2:")
print(union_group)  # ➡️ [Ana]->[Luis]->[Marta]->[Carlos]->[Elena]->None

# Probar que no se agregan duplicados
group1.add_member("Luis")  # Ya está
print("\nGrupo1 tras intentar añadir a 'Luis' otra vez:")
print(group1)  # ➡️ [Ana]->[Luis]->[Marta]->None

# Probar que no se elimnan membros
group1.remove_member("Marta")  # Ya está
print("\nGrupo1 tras eliminar 'Marta':")
print(group1)  # ➡️ [Ana]->[Luis]->None

```

## networking.py

```python
from containerbase import ContainerBase

class InterestGroup(ContainerBase):

    class Node:
        def __init__(self, value, next=None):
            self.value = value
            self.next = next

    def __init__(self):
        self.first = None

```

VPL